

Assignment-11.1

Name: Sushma Purella

H.No:2303A52188

Batch:44

Task Description #1– Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods

Prompt:

Use AI to generate a Python class that implements a Stack data structure with the following methods:

- push(item) – Add an element to the stack
- pop() – Remove and return the top element
- peek() – Return the top element without removing it
- is_empty() – Check whether the stack is empty

The stack should follow the LIFO (Last In First Out) principle and handle empty stack conditions safely.

Code:

```
Assignment-11.1.py > ...
1  #Use AI to generate a Stack class with push, pop, peek, and is_empty methods.
2  class Stack:
3      def __init__(self):
4          self.items = []
5      def push(self, item):
6          self.items.append(item)
7      def pop(self):
8          if not self.is_empty():
9              return self.items.pop()
10         return None
11     def peek(self):
12         if not self.is_empty():
13             return self.items[-1]
14         return None
15     def is_empty(self):
16         return len(self.items) == 0
17 stack = Stack()
18 stack.push(1)
19 stack.push(2)
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
Attachments\Desktop\Ai-Assist\Lab test-q1.py
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py
None
True
```

```
20     assert stack.peek() == 2
21     assert stack.pop() == 2
22     assert stack.peek() == 1
23     assert not stack.is_empty()
24     stack.pop()
25     assert stack.is_empty()
26     print(stack.pop())
27     print(stack.is_empty())
28
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py
None
True
```

Observation:

The implemented Stack class correctly follows the LIFO principle, where the last element pushed is the first to be removed. The push() method adds elements, while pop() and peek() safely handle empty

stack conditions by returning None when the stack is empty. The `is_empty()` method efficiently checks stack status using length comparison. Overall, the implementation demonstrates proper stack behavior using Python lists

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Prompt

Use AI to generate a Python class that implements a Queue data structure with the following methods:

- `enqueue(item)` – Add an element to the rear of the queue
- `dequeue()` – Remove and return the front element
- `peek()` – Return the front element without removing it
- `is_empty()` – Check whether the queue is empty

The queue should follow the FIFO (First In First Out) principle and handle empty queue conditions safely.

Code:

```
31 #implement a Queue using Python lists.
32 class Queue:
33     def __init__(self):
34         self.items = []
35     def enqueue(self, item):
36         self.items.append(item)
37     def dequeue(self):
38         if not self.is_empty():
39             return self.items.pop(0)
40         return None
41     def peek(self):
42         if not self.is_empty():
43             return self.items[0]
44         return None
45     def is_empty(self):
46         return len(self.items) == 0
47 queue = Queue()
48 queue.enqueue(1)
49 queue.enqueue(2)
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py

None
True

```
47 queue = Queue()
48 queue.enqueue(1)
49 queue.enqueue(2)
50 assert queue.peek() == 1
51 assert queue.dequeue() == 1
52 assert queue.peek() == 2
53 assert not queue.is_empty()
54 queue.dequeue()
55 assert queue.is_empty()
56 print(queue.dequeue())
57 print(queue.is_empty())
58
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py

None
True

Observation:

The implemented Queue class correctly follows the FIFO principle, where the first element inserted is the first one removed. The enqueue() method adds elements at the end, and dequeue() removes elements from the front using pop(0). The peek() method returns the

front element without removal, and `is_empty()` checks whether the queue contains elements. The implementation works correctly, though using `pop(0)` has $O(n)$ time complexity because elements are shifted after removal.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Prompt

Use AI to generate a Python program that implements a Singly Linked List with the following:

- A Node class containing data and next pointer
- An `insert(data)` method to add elements at the end of the list
- A `display()` method to print all elements in the list

The linked list should dynamically store elements and maintain proper node connections.

Code:

```
61 #generate a Singly Linked List with insert and display methods
62 class Node:
63     def __init__(self, data):
64         self.data = data
65         self.next = None
66 class SinglyLinkedList:
67     def __init__(self):
68         self.head = None
69     def insert(self, data):
70         new_node = Node(data)
71         if not self.head:
72             self.head = new_node
73             return
74         last_node = self.head
75         while last_node.next:
76             last_node = last_node.next
77         last_node.next = new_node
78     def display(self):
79         current_node = self.head
80         while current_node:
81             print(current_node.data, end=' ')
82             current_node = current_node.next
83         print()
84 linked_list = SinglyLinkedList()
85 linked_list.insert(1)
86 linked_list.insert(2)
87 linked_list.insert(3)
88 linked_list.display() # Output: 1 2 3
89
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

1 2 3

Observation:

The implemented Singly Linked List correctly maintains dynamic connections between nodes using pointers. The insert() method adds new elements at the end of the list by traversing to the last node and updating its next reference. The display() method traverses the list from head to tail and prints each element. This implementation demonstrates efficient dynamic memory usage without requiring contiguous storage like arrays.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Prompt:

Use AI to generate a Python program that implements a Binary Search Tree (BST) with the following features:

- A Node class containing data, left, and right pointers
- An insert(data) method to add elements while maintaining BST properties
- An in_order_traversal() method to display elements in sorted order

The BST should follow the rule:

Left subtree < Root < Right subtree.

Code:

```
90
91 #create a BST with insert and in-order traversal methods.
92 class Node:
93     def __init__(self, data):
94         self.data = data
95         self.left = None
96         self.right = None
97 class BinarySearchTree:
98     def __init__(self):
99         self.root = None
100     def insert(self, data):
101         if self.root is None:
102             self.root = Node(data)
103         else:
104             self._insert_recursive(self.root, data)
105     def _insert_recursive(self, node, data):
106         if data < node.data:
107             if node.left is None:
108                 node.left = Node(data)
109             else:
110                 self._insert_recursive(node.left, data)
111         else:
112             if node.right is None:
113                 node.right = Node(data)
114             else:
115                 self._insert_recursive(node.right, data)
116     def in_order_traversal(self):
117         return self._in_order_recursive(self.root)
118     def _in_order_recursive(self, node):
```

```
113         node.right = Node(data)
114     else:
115         self._insert_recursive(node.right, data)
116     def in_order_traversal(self):
117         return self._in_order_recursive(self.root)
118     def _in_order_recursive(self, node):
119         res = []
120         if node:
121             res = self._in_order_recursive(node.left)
122             res.append(node.data)
123             res = res + self._in_order_recursive(node.right)
124         return res
125 bst = BinarySearchTree()
126 bst.insert(5)
127 bst.insert(3)
128 bst.insert(7)
129 print(bst.in_order_traversal())
130
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python310/python.exe C:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py
Processing Order: 101
Pending Orders: [102, 103]
```

Observation:

The implemented Binary Search Tree correctly maintains the BST property during insertion by placing smaller values in the left subtree and larger values in the right subtree. The recursive insertion method ensures proper node placement. The `in_order_traversal()` method visits nodes in Left → Root → Right order, producing a sorted list of elements. This demonstrates how BST enables efficient searching and sorted data retrieval.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods

Prompt:

Use AI to generate a Python program that implements a Hash Table with the following methods:

- `insert(key, value)` – Store a key-value pair

- search(key) – Retrieve value using key
- delete(key) – Remove a key-value pair

The hash table should allow fast lookup and handle cases where the key does not exist.

Code:

```

91
92 #mplement a hash table with basic insert, search, and delete methods.
93 class HashTable:
94     def __init__(self):
95         self.table = {}
96     def insert(self, key, value):
97         self.table[key] = value
98     def search(self, key):
99         return self.table.get(key, None)
100     def delete(self, key):
101         if key in self.table:
102             del self.table[key]
103 hash_table = HashTable()
104 hash_table.insert("name", "Alice")
105 hash_table.insert("age", 30)
106 assert hash_table.search("name") == "Alice"
107 assert hash_table.search("age") == 30
108 hash_table.delete("name")
109 assert hash_table.search("name") is None
110 print(hash_table.search("age"))
111 print(hash_table.search("name"))
112 |

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py

30
None

Observation:

The implemented Hash Table uses Python's built-in dictionary to store key-value pairs efficiently. The insert() method adds or updates entries, search() retrieves values in average $O(1)$ time, and delete() removes entries if they exist. The implementation demonstrates how hashing enables fast data access and efficient key-based storage.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Prompt:

Use AI to generate a Python program that implements a **Graph** using an **Adjacency List** representation.

The graph should include:

- `add_vertex(vertex)` – Add a new vertex
- `add_edge(vertex1, vertex2)` – Add an undirected edge between two vertices
- `display()` – Print all vertices and their connected edges

The graph should dynamically store connections between vertices.

Code:

```
115 # implement a graph using an adjacency list.
116 class Graph:
117     def __init__(self):
118         self.adjacency_list = {}
119     def add_vertex(self, vertex):
120         if vertex not in self.adjacency_list:
121             self.adjacency_list[vertex] = []
122     def add_edge(self, vertex1, vertex2):
123         if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
124             self.adjacency_list[vertex1].append(vertex2)
125             self.adjacency_list[vertex2].append(vertex1)
126     def display(self):
127         for vertex, edges in self.adjacency_list.items():
128             print(f"{vertex}: {edges}")
129 graph = Graph()
130 graph.add_vertex("A")
131 graph.add_vertex("B")
132 graph.add_vertex("C")
133 graph.add_edge("A", "B")
134 graph.add_edge("A", "C")
135 graph.display()
136
137
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py
A: ['B', 'C']
B: ['A']
C: ['A']
```

Observation:

The implemented Graph uses a dictionary to represent the adjacency list, where each vertex maps to a list of connected vertices. The `add_vertex()` method ensures vertices are created before edges are added, and `add_edge()` connects two vertices in both directions, forming an undirected graph. This structure efficiently represents relationships between nodes and is space-efficient for sparse graphs.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's `heapq` module.

Prompt:

Use AI to generate a Python program that implements a Priority Queue using Python's built-in `heapq` module.

The class should include:

- `push(item, priority)` – Insert an element with a given priority
- `pop()` – Remove and return the element with the highest priority (lowest priority number in min-heap)
- `peek()` – View the highest priority element without removing it
- `is_empty()` – Check whether the priority queue is empty

The implementation should follow the heap property for efficient priority-based retrieval.

Code:

```
138
139 #implement a priority queue using Python's heapq module.
140 import heapq
141 class PriorityQueue:
142     def __init__(self):
143         self.elements = []
144     def push(self, item, priority):
145         heapq.heappush(self.elements, (priority, item))
146     def pop(self):
147         return heapq.heappop(self.elements)[1] if self.elements else None
148     def peek(self):
149         return self.elements[0][1] if self.elements else None
150     def is_empty(self):
151         return len(self.elements) == 0
152 pq = PriorityQueue()
153 pq.push("task1", priority=2)
154 pq.push("task2", priority=1)
155 assert pq.peek() == "task2"
156 assert pq.pop() == "task2"
157 assert pq.peek() == "task1"
158 assert not pq.is_empty()
159 pq.pop()
160 assert pq.is_empty()
161 print(pq.pop())
162 print(pq.is_empty())
163
164
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py
None
True
```

Observation:

The implemented Priority Queue uses Python's heapq, which internally maintains a min-heap structure. Elements are stored as (priority, item) tuples, ensuring that the item with the smallest priority value is removed first. The push() and pop() operations run in $O(\log n)$ time, while peek() operates in $O(1)$ time. This implementation efficiently manages tasks based on priority rather than insertion order.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Prompt:

Use AI to generate a Python program that implements a Double-Ended Queue (Deque) using Python's built-in collections.deque module.

The class should include:

- `append(item)` – Add element to the right end
- `appendleft(item)` – Add element to the left end
- `pop()` – Remove and return element from the right end
- `popleft()` – Remove and return element from the left end
- `peek()` – View the front element
- `is_empty()` – Check whether the deque is empty

The implementation should allow insertion and deletion from both ends efficiently.

Code:

```
165 #implement a double-ended queue using collections.deque
166 from collections import deque
167 class Deque:
168     def __init__(self):
169         self.deque = deque()
170     def append(self, item):
171         self.deque.append(item)
172     def appendleft(self, item):
173         self.deque.appendleft(item)
174     def pop(self):
175         return self.deque.pop() if self.deque else None
176     def popleft(self):
177         return self.deque.popleft() if self.deque else None
178     def peek(self):
179         return self.deque[0] if self.deque else None
180     def is_empty(self):
181         return len(self.deque) == 0
182 dq = Deque()
183 dq.append("item1")
184 dq.appendleft("item2")
185 assert dq.peek() == "item2"
186 assert dq.pop() == "item1"
187 assert dq.peek() == "item2"
188 assert not dq.is_empty()
189 dq.popleft()
190 assert dq.is_empty()
191 print(dq.pop())
192 print(dq.is_empty())
193
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py
None
True
```

Observation:

The implemented Deque uses Python's `collections.deque`, which allows efficient insertion and deletion from both ends in $O(1)$ time.

The `append()` and `appendleft()` methods add elements to the rear and front respectively, while `pop()` and `popleft()` remove elements from both ends. The `peek()` method retrieves the front element without removing it. This structure is flexible and combines features of both Stack and Queue.

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System

that handles:

- 1. Student Attendance Tracking – Daily log of students entering/exiting the campus.**
- 2. Event Registration System – Manage participants in events with quick search and removal.**
- 3. Library Book Borrowing – Keep track of available books and their due dates.**
- 4. Bus Scheduling System – Maintain bus routes and stop connections.**
- 5. Cafeteria Order Queue – Serve students in the order they arrive.**

Prompt:

Design and implement a Campus Resource Management System that efficiently manages multiple campus operations using appropriate data structures.

Each feature must use a suitable data structure based on its

operational requirements such as fast search, insertion, deletion, ordering, or connectivity handling.

The system includes:

1. Student Attendance Tracking
2. Event Registration System
3. Library Book Borrowing
4. Bus Scheduling System
5. Cafeteria Order Queue

Each module is implemented independently using the most efficient data structure for that task.

Code:

```
195
196 class Attendance:
197     def __init__(self):
198         self.record = {} # student_id : status
199
200     def entry(self, student_id):
201         self.record[student_id] = "IN"
202
203     def exit(self, student_id):
204         self.record[student_id] = "OUT"
205
206     def status(self, student_id):
207         return self.record.get(student_id, "Not Found")
208 a = Attendance()
209 a.entry(101)
210 a.exit(101)
211 print(a.status(101))
212
213
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

hon313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py

OUT

```

242
243 class Library:
244     def __init__(self):
245         self.books = {}
246
247     def borrow(self, book_id, due_date):
248         self.books[book_id] = due_date
249
250     def return_book(self, book_id):
251         if book_id in self.books:
252             del self.books[book_id]
253
254     def check(self, book_id):
255         return self.books.get(book_id, "Available")
256
257 lib = Library()
258 lib.borrow(1, "10 Feb")
259 print(lib.check(1))

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py
10 Feb

```

```

260
261 class BusGraph:
262     def __init__(self):
263         self.graph = {}
264
265     def add_route(self, stop1, stop2):
266         if stop1 not in self.graph:
267             self.graph[stop1] = []
268         if stop2 not in self.graph:
269             self.graph[stop2] = []
270         self.graph[stop1].append(stop2)
271         self.graph[stop2].append(stop1)
272
273     def display(self):
274         print(self.graph)
275
276 bus = BusGraph()
277 bus.add_route("StopA", "StopB")
278 bus.add_route("StopB", "StopC")
279 bus.display()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py
{'StopA': ['StopB'], 'StopB': ['StopA', 'StopC'], 'StopC': ['StopB']}

```



```
280 from collections import deque
281 class Cafeteria:
282     def __init__(self):
283         self.queue = deque()
284
285     def order(self, student):
286         self.queue.append(student)
287
288     def serve(self):
289         if self.queue:
290             return self.queue.popleft()
291         return "No Orders"
292
293 cafe = Cafeteria()
294 cafe.order("Sushma")
295 cafe.order("Arjun")
296 print(cafe.serve())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS + v

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py

Sushma

Observation:

The system demonstrates how different real-world problems require different data structures for optimal performance.

- Hash Tables provide constant-time access for tracking attendance and books.
- Binary Search Tree enables structured storage with efficient search and removal.
- Graph effectively models interconnected bus routes.
- Queue ensures fair servicing in cafeteria using FIFO principle.

This implementation shows that choosing the correct data structure improves efficiency, maintainability, and scalability of a system.

Task Description #10: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System

with:

- 1. Shopping Cart Management – Add and remove products dynamically.**
- 2. Order Processing System – Orders processed in the order they are placed.**
- 3. Top-Selling Products Tracker – Products ranked by sales count.**
- 4. Product Search Engine – Fast lookup of products using product ID.**
- 5. Delivery Route Planning – Connect warehouses and delivery locations.**

Prompt:

Design a Smart E-Commerce Platform that efficiently manages shopping operations using suitable data structures. Each feature must use the most appropriate data structure based on operational requirements such as dynamic insertion, ranking, fast searching, order processing, and network connectivity.

Code:

```

297
298
299 from collections import deque
300 class OrderProcessingSystem:
301     def __init__(self):
302         self.orders = deque()
303     def place_order(self, order_id):
304         self.orders.append(order_id)
305         print(f"Order {order_id} placed successfully.")
306     def process_order(self):
307         if self.orders:
308             processed = self.orders.pop() (variable) processed: Any
309             print(f"Processing Order: {processed}")
310         else:
311             print("No orders to process.")
312     def display_orders(self):
313         """Display all pending orders."""
314         if self.orders:
315             print("Pending Orders:", list(self.orders))
316         else:
317             print("No pending orders.")
318 system = OrderProcessingSystem()
319 system.place_order(101)
320 system.place_order(102)
321 system.place_order(103)
322 system.display_orders()
323 system.process_order()
324 system.display_orders()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-11.1.py
Processing Order: 101
Pending Orders: [102, 103]

```

Observation:

Different modules in an e-commerce system require different data structures to achieve efficiency. Queue ensures fair order processing, Linked List allows flexible cart updates, Priority Queue helps in ranking top-selling products, Hash Table enables instant product lookup, and Graph models delivery routes effectively. Selecting the correct data structure improves system performance and scalability.